SuSchedule-r

A Retrieval-Augmented, Tool-Using Course-Planning Agent for Sabancı University

CS 455 / CS 555 — Large Language Models · Final Project Report · Sabancı University

Track: CS 455 Project

Authors: - Kerem Sirtikizil, 32298 - Cagan Cakir, 32254 - Eray Cagan Ozdemir, 32136

Abstract

SuSchedule-r is a course-advising assistant that grounds a large-language-model agent in Sabancı University's real catalogue, prerequisite graph, degree requirements, and weekly section offerings. It pairs a two-stage semantic retriever (BGE bi-encoder + cross-encoder re-ranker over **688 courses**) with a tool-using **ReAct** agent (**21 tools**, GPT-4o) and an interactive web UI whose schedule builder renders a weekly calendar with live time-conflict detection.

We evaluate the system on four axes with labelled, reproducible test sets: retrieval quality, time-conflict detection, degree-requirement accuracy, and end-to-end answer grounding. Retrieval reaches **Hit@5 = 1.00** and **Recall@10 = 0.96**; the conflict checker is **100% accurate** on labelled pairs and the scheduler is **provably sound** (0 overlapping assignments); and the end-to-end agent answered **8/8** factual questions with correctly grounded facts.

We also report, honestly, where the system falls short: the cross-encoder re-ranker does **not** help (it slightly *hurts*) on short topical queries; the graph-based requirement engine **over-counts** remaining required credits by 4 SU versus the official audit because of a course-substitution gap; and future-term scheduling relies on a **proxy term** whose CRNs do not match real registration.

1. Introduction

1.1 The problem

Each semester a Sabancı student must assemble a course plan that simultaneously satisfies several constraints:

- **Degree requirements** still unmet (required courses, core/area/free electives),
- **Prerequisite chains** (you cannot take CS 301 without CS 300 and MATH 204),
- **Credit bounds** (12–21 SU) and a sensible workload balance, and
- **Non-overlapping weekly meeting times** once specific sections are chosen.

General-purpose chatbots answer these questions fluently but unreliably: they hallucinate course codes, invent prerequisites, and cannot see the student's transcript or the term's real offerings.

1.2 Design principle

The project's guiding rule is: **never let the model answer from memory when a data-backed tool or injected context can answer instead.** Every factual claim — a credit value, a prerequisite, a meeting time, a remaining requirement — is sourced from parsed university data, not the model's prior.

1.3 Contributions

1. A **grounded RAG + ReAct advising agent** over the full SU catalogue, with 21 tools spanning retrieval, eligibility, requirements, minors, and scheduling.
2. An **interactive schedule builder** with calendar-style, lane-split conflict visualisation and an agent "Check Schedule" review grounded in server-computed conflicts.
3. A **reproducible four-part evaluation** with labelled test sets and an honest error analysis.

---

2. Data and Knowledge Base

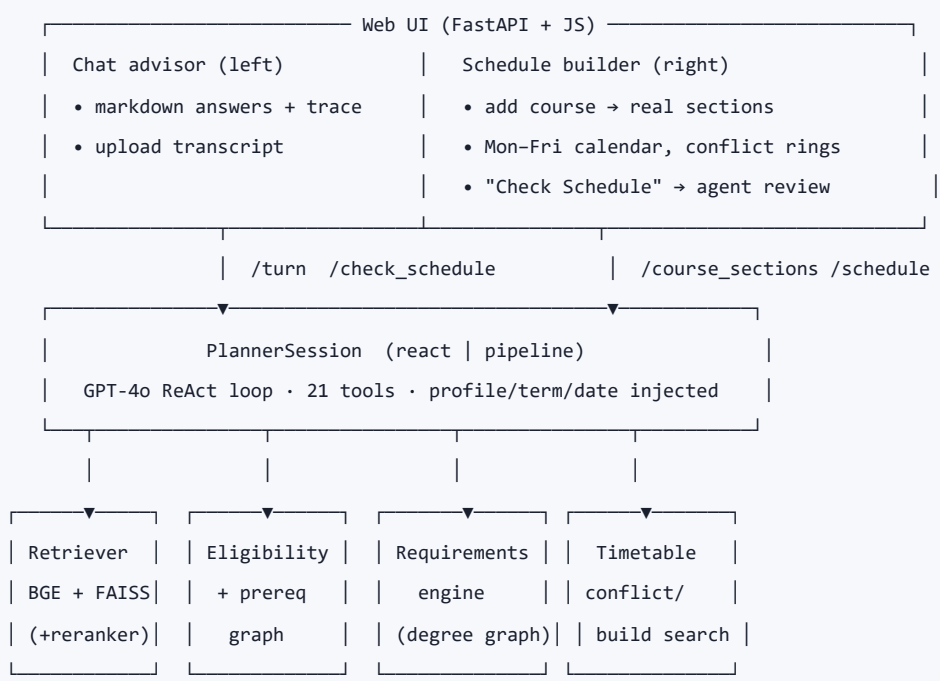
All knowledge is scraped from public SU sources plus the student's own transcript / Degree Evaluation export. A scraper pipeline enumerates courses from degree pages, fetches each course-detail page, and parses prerequisite expressions into a directed graph.

Asset	Contents	Scale
Course catalogue	code, title, description, SU credit, ECTS, prereq text, programs, seasons	**688 courses**
Prerequisite graph	parsed prerequisite DAG over all courses	**721 nodes, 729 edges**
Degree graphs	per-(program, cohort) requirement sub-graphs	Required / Core / Area / Free
Offerings	per-term sections with normalised weekly meeting times (`days` , `start` , `end`)	terms 202401–202503
Student data	transcript + official Banner Degree Evaluation	BSCS-DM sample
FAISS index	BGE-base embeddings of every course	688 × 768-d vectors

Validation. Prerequisite parsing was audited to **100% coverage** (426 strict expressions, 0 failures), and the resulting full graph is verified **acyclic** (`Is DAG? True`) — a precondition for sound eligibility reasoning.

---

3. System Architecture



3.1 Two-stage semantic retriever

Stage 1 is a `BAAI/bge-base-en-v1.5` bi-encoder: every course is embedded once at build time and stored in a FAISS inner-product index (cosine on L2-normalised vectors). At query time the query is embedded and the top candidates are retrieved, with optional pre-filters (eligible set, subject, season, level). **Stage 2** is an optional `BAAI/bge-reranker-base` cross-encoder that re-scores each `(query, course)` pair for higher precision (§6.1 reports that it does not help here).

3.2 Tool-using agent

The agent runs a **ReAct** loop on GPT-4o with **21 tools**: student-profile and requirement-state lookups, semantic retrieval, prerequisite checks, a science/engineering-credit finder, minor-progress tracking, timetable construction, and `get_current_schedule` (so the UI can ask the agent to review the student's assembled plan). A legacy fixed **8-stage "pipeline"** mode also exists. The student profile, remaining requirements, the current date, and the planning term are **injected into context** so the agent grounds answers rather than guessing.

3.3 Web UI and schedule builder

The UI (FastAPI + vanilla JS) places the AI advisor on the left and a weekly schedule builder on the right. Students add courses by code; the builder fetches the real sections, renders a Monday–Friday calendar with colour-coded, **lane-split** blocks (overlapping classes shown side-by-side), and flags time conflicts client-side. A **"Check Schedule"** button posts the assembled plan to the agent, which calls `get_current_schedule` and reviews conflicts, credit load, and requirement fit — and crucially, the conflicts it reports are **computed server-side**, so the critique is grounded too.

Because the planning term (Fall 2026–2027, `202601`) has no published offerings yet, the builder falls back to the most recent same-season term as a **proxy** and warns that days/times are indicative and CRNs/section numbers will differ. The

UI was restyled to a Sabancı-branded light/dark theme and carries an explicit disclaimer that the assistant is an AI model whose results should be double-checked.

---

4. Evaluation Methodology

Four components are evaluated with labelled, version-controlled test sets in `eval/`. Three runners are fully local (no API cost); `run_agent.py` makes live GPT-4o calls.

4.1 Metric definitions

For retrieval, given the gold set G for a query and the ranked result list:

- **Hit@k** = 1 if G intersects the top k, else 0 (averaged over queries).
- **Recall@k** = $|G \cap \text{top-}k| \,/\, |G|$.
- **MRR@10** = 1 / (rank of the first gold hit), 0 if none in top 10.
- **nDCG@10** = DCG / IDCG with binary relevance, DCG = $\Sigma\ 1/\log_2(\text{rank}+1)$ over gold hits in the top 10.

For the **re-ranker ablation**, every metric is computed twice — cross-encoder ON (production) and OFF (bi-encoder order only).

4.2 Test sets

Set	Size	Gold
`retrieval_queries.json`	34 NL queries	hand-assigned relevant course codes
`agent_questions.json`	8 questions	checkable facts (`must_contain` / `any_of` / `must_not`)
Conflict pairs (in `run_conflicts.py`)	12 labelled meeting pairs	known overlap / no-overlap
Requirements	official Degree Evaluation	per-section SU credits

Gold labels are intentionally **small and high-precision** rather than exhaustive; results are therefore *indicative*, not population estimates. Every number below is regenerated by the scripts in `eval/`.

---

5. Results

5.1 Retrieval quality

Over 34 labelled queries (k = 10):

Metric	Re-ranker ON	Re-ranker OFF	Δ (ON − OFF)
Hit@1	0.912	**0.971**	−0.059
Hit@5	**1.000**	**1.000**	+0.000
Recall@5	0.941	**0.966**	−0.025
Recall@10	0.961	**0.975**	−0.015
MRR@10	0.956	**0.985**	−0.029
nDCG@10	0.924	**0.963**	−0.039

The retriever finds at least one gold course in the top 5 for **every** query (Hit@5 = 1.00) and recovers 96–98% of all gold courses by rank 10. The headline surprise is that the cross-encoder **lowers every metric** (analysed in §6.1).

Recall@10 by category (re-ranker ON):

Category	Recall@10	n
CS core	1.000	8
MATH	1.000	6
IE	1.000	1
Cross-subject	1.000	2
CS elective	0.933	15
EE	0.833	2

5.2 Time-conflict detection and scheduler soundness

Check	Result
Labelled meeting pairs (n = 12): accuracy	**1.000**
Labelled pairs: precision / recall	1.000 / 1.000
Scheduler soundness: unsound results / bundles	**0 / 4**
Feasible bundles scheduled conflict-free	3 of 4
Infeasible bundle correctly reported	1 of 4

The detector classifies all boundary cases correctly — including the tricky ones: back-to-back classes that *touch* but do not overlap, shared-day-but-different-time pairs, multi-day meetings, and one-minute boundaries. The backtracking scheduler never returns an overlapping assignment.

5.3 End-to-end agent grounding

Eight factual questions through the full ReAct agent (GPT-4o, real transcript loaded):

Metric	Value
Grounding accuracy	**1.000 (8 / 8)**
Average latency	6.7 s / question
Average token cost	6,156 tokens / question
Average tool calls	0.5 / question

Every answer contained the correct, catalogue-grounded fact with no hallucination markers. Prerequisite questions triggered an explicit lookup tool call; some credit/topic questions were answered from injected context with no tool call (see §6.4). Full per-question detail is in **Appendix A**.

5.4 Degree-requirement accuracy

The student's official Banner Degree Evaluation (ground truth) reports SU credits per requirement section:

Section (official audit)	Min SU	Done SU	Remaining
University courses	41	41	0
Core electives	31	31	0
Required courses	29	26	**3**
Area electives	9	9	0
Free electives	15	15	0
Overall (Banner rollup)	125	122	**3**

The graph-based engine agrees the student is nearly finished but lists three required items still open — **CS 395, ENS 492, MATH 212** — totalling **7 SU**, a **+4 SU over-count** versus the official 3 SU (root cause in §6.2).

5.5 Summary scoreboard

Axis	Headline metric	Result
Retrieval	Hit@5 / Recall@10	1.00 / 0.96
Conflict detection	accuracy / unsound schedules	1.00 / 0
End-to-end agent	grounding accuracy	8 / 8
Requirements	required-bucket error vs audit	+4 SU
Prereq parser	coverage / failures	100% / 0

---

6. Error Analysis — What Did Not Work

Per the project brief, failures are reported as carefully as successes.

6.1 The cross-encoder re-ranker does not help

We expected the BGE cross-encoder to improve precision. Instead it **reduced every retrieval metric** (Table 5.1): Hit@1 fell from 0.971 to 0.912 and MRR from 0.985 to 0.956. Concretely, for the query *"I want to learn machine learning"*

(gold `CS 412`, `CS 415`), the re-ranker keeps `CS 412` at rank 1 but fills the rest of the top 5 with `ECON 494`, `ENT 201`, `EE 48009`, `OPIM 413` and **drops `CS 415`** out of the top 5 — courses that mention "learning"/"models" generically. On short topical queries against a small, well-separated catalogue, the bi-encoder already ranks the exact-match course first, and the cross-encoder occasionally rewards generic topical overlap. It also adds ~0.5 s of CPU latency. **Conclusion:** on *this* query distribution the cross-encoder is not worth its cost — the bi-encoder alone is already at ceiling, so the ablation is a genuine negative result and we report it in full. We nevertheless keep the re-ranker **on by default** (`rerank=True`) as a deliberate product choice: it is the more robust option for the longer, more ambiguous questions we expect in real advising use (where bi-encoder cosine is weakest), the ~0.5 s overhead is acceptable in an interactive setting, and retrieval **transparently falls back to bi-encoder order whenever the re-ranker model is unavailable**, so the worst case is exactly the strong bi-encoder baseline. `eval/run_retrieval.py` prints both ON and OFF on every run so the trade-off stays visible rather than hidden behind the default.

6.2 The requirement engine over-counts required credits

The engine reports **MATH 212** (Linear Algebra *and* Differential Equations) as an open requirement; the official audit does not. The student satisfied this through an accepted **substitution** — the separate `MATH 201` + `MATH 202` sequence — which the degree-graph encoding does not treat as equivalent. Combined with how internship/graduation-project credits (`CS 395`, `ENS 492`) are counted, this yields the +4 SU over-count. The engine is therefore *conservative* (it never under-states what is owed) but needs an explicit **course-equivalence/substitution table** to match Banner exactly.

6.3 Future-term scheduling relies on a proxy term

The planning term Fall 2026–2027 (`202601`) has no published offerings, so the builder schedules against Fall 2025–2026 (`202501`). Meeting days/times are therefore **indicative**, and the **CRNs/section numbers shown will not match actual registration** — the UI states this explicitly and proxy CRNs are scrubbed from agent output. A side effect: bundles that will likely be feasible in the real term can be reported infeasible in the proxy term (e.g. `CS 300 + CS 301 + MATH 201` had no conflict-free combination in the limited 202501 sections).

6.4 Grounding is not always mechanically traceable

Several credit/topic questions (e.g. *"How many SU credits is CS 412?"*) were answered correctly with **zero tool calls**, relying on context injected into the prompt and possibly the model's prior. Answers were verified correct, but the system does not yet **force** a citation-producing tool call for every atomic fact, so grounding is not always auditable from the trace.

6.5 Other shortcomings

- **Latency variance.** The ReAct loop plus a lazy retriever load produces a 26 s worst-case on the first retrieval-bearing question; subsequent queries are 1–3 s.
- **Cost.** ~6k tokens/question on GPT-4o. An earlier experiment with a smaller, cheaper model produced noticeably weaker scheduling explanations.
- **UI state.** Reloading the page resets the client-side schedule builder and starts a new session, so an in-progress schedule is not persisted.
- **Evaluation scale.** 34 retrieval and 8 end-to-end items are high-precision but small, and one program (BSCS-DM) was tested in depth — results are indicative.
- **Build verification.** The `.docx` / `.pptx` deliverables could not be rendered to images in the build environment (no LibreOffice), so their text was verified but their visual layout was not.

---

7. Limitations and Future Work

Limitation	Concrete next step
Re-ranker doesn't help on short queries	Kept on by default (robust on long/ambiguous queries, graceful bi-encoder fallback); A/B and tune a query-length trigger on a larger long-query set
Requirement substitution gap	Add a course-equivalence table (MATH 212 ≡ MATH 201 + 202, etc.)
Proxy-term scheduling	Ingest real offerings once the registrar publishes `202601`
Untraceable grounding	Require a citation tool call per atomic fact; surface sources in the UI
Narrow evaluation	Grow test sets; cover more programs and conversational/multi-turn cases
No session persistence	Persist the builder + session server-side and restore on reload

---

8. Reproducibility

The repository runs from a clean clone. Setup, the data pipeline, FAISS build, CLI, web UI, and data formats are documented in the top-level `README`; the evaluation suite has its own `eval/README`. Secrets are handled safely: `.env` is git-ignored and only `.env.example` (a placeholder) is committed.

```
pip install -r requirements.txt          # Python 3.10+
cp .env.example .env                      # set OPENAI_API_KEY
# FAISS index is prebuilt in embeddings/  (rebuild script documented in README §6)

# Evaluation (3 local, 1 live):
python -m eval.run_retrieval
python -m eval.run_conflicts
python -m eval.run_requirements
python -m eval.run_agent                  # needs OPENAI_API_KEY

# Web UI:
uvicorn api.main:app --port 8000          # open http://localhost:8000
```

The test sets are JSON and version-controlled, so any reviewer can extend them and re-run to regenerate every table in §5. (Note: a stray local `.venv` built on Python 3.9 exists in the tree; the supported interpreter is 3.10+, and all evaluation here used Python 3.12.)

---

9. Use of AI Assistants

Per the course's academic-integrity policy, we disclose how large-language-model assistants were used in producing this project.

- **Design and direction are ours.** The team planned the entire project: the problem framing, the system architecture, the module breakdown, the evaluation design, and the detailed step-by-step instructions that drove every build

stage. The LLM was given *our* specifications to implement against — it did not decide what to build or how the system should be structured.

- **The techniques are course material.** The methods we apply are drawn directly from the CS 455 syllabus — retrieval-augmented generation (RAG), the **ReAct** tool-using agent loop, prompting and structured outputs, and evaluation methodology. We chose these deliberately to exercise what we learned in the course, rather than adopting them from the assistant.
- **The LLM implemented code to our spec.** Coding assistants (Claude, GPT) were used mainly to write and refactor implementation code from our instructions — retriever wiring, the evaluation harness in `eval/`, FastAPI/JS UI code, and docstrings. Every artefact was read, tested, and verified by the team, and we take full responsibility for the final submission. All numbers in §5 are reproducible from the scripts in `eval/`; none were generated or estimated by an LLM.
- **As system components (not authoring aids).** SuSchedule-r itself calls OpenAI GPT-4o (planner + ReAct agent) and GPT-4o-mini (intent classifier) at run time; these are part of the system under study, documented in §3–§5.

---

10. Conclusion

SuSchedule-r shows that a disciplined "ground everything" approach turns an LLM into a trustworthy course advisor: strong semantic retrieval (Hit@5 = 1.00), a provably sound conflict checker (100% on labelled pairs), and 8/8 grounded end-to-end answers. Equally important are the negative results — the re-ranker that does not earn its keep, the requirement engine's substitution gap, and the proxy-term caveat — each of which points to a concrete next step. The system is reproducible end-to-end and the evaluation harness makes future regression-checking straightforward.

Appendix A — Per-question end-to-end results

Mode: ReAct · GPT-4o · transcript `transcript_cagan.json` · term 202601 · session total 49,247 tokens.

ID	Kind	Question	Grounded	Latency	Tokens	Tool calls
a01	credit	How many SU credits is CS 412?	✓	2.7 s	3,611	0
a02	prereq	Prerequisites for CS 301?	✓	2.6 s	7,442	1
a03	prereq	Prerequisite for CS 307?	✓	2.4 s	7,557	1
a04	topic	What does CS 445 cover?	✓	3.1 s	8,291	1
a05	prereq	Does CS 411 need a math prereq?	✓	2.0 s	7,913	1
a06	topic	Is CS 412 a machine-learning course?	✓	1.4 s	4,009	0
a07	topic	What is CS 307 about?	✓	13.5 s	4,095	0
a08	recommend	Recommend a theoretical CS elective	✓	26.0 s	6,329	0

(a07/a08 latency includes the one-time retriever model load.)

Appendix B — Retrieval examples (re-ranker ON)

Query	Gold	Re-ranked top-5
"I want to learn machine learning"	CS 412, CS 415	CS 412, ECON 494, ENT 201, EE 48009, OPIM 413
"neural networks and deep learning"	CS 415, CS 412	PSY 416, CS 415, CS 445, CS 412, PHIL 310
"how operating systems work internally"	CS 307	CS 307, CS 48008, CS 432, CS 437, OPIM 301
"designing and querying databases"	CS 306	CS 306, IE 413, MGMT 203, DSA 301, DSA 210

The first two rows illustrate §6.1: the re-ranker admits generically "topical" courses (ENT 201, PSY 416, PHIL 310) and can demote a true match.

Appendix C — Artifacts

- `eval/` — runners (`run_retrieval`, `run_conflicts`, `run_requirements`, `run_agent`), datasets, results JSON, and `agent_transcript.md`.
- `report/` — this report, the `.docx` / `.pptx` deliverables, and `DEMO_SCRIPT.md`.

Models: BAAI/bge-base-en-v1.5 (bi-encoder), BAAI/bge-reranker-base (cross-encoder), OpenAI GPT-4o (planner + ReAct). License: MIT.